HSLU Hochschule Luzern

# I.BA_CSA
# C# in Action

# Anhang Http

Roger Diehl, Luzern
© Copyright 2026

# Hypertext Transfer Protocol

(Autor: Holger Reibold, angepasst von P. Infanger / R. Diehl)



HTML Erfinder Tim Berners-Lee hat nicht nur HTML, sondern auch einen Transportmechanismus für die plattformübergreifenden Dokumente entwickelt: HTTP ist das wichtigste Transportprotokoll für webbasierte Inhalte.

Tim Berners-Lee
http://www.w3.org/People/Berners-Lee

Der Erfolg des World Wide Webs ist zu einem großen Teil auf die Einfachheit von HTTP zurückzuführen. Das Protokoll lässt sich vergleichsweise einfach implementieren. Das zu Grunde liegende Prinzip ist simpel: Die Übermittlung der Daten erfolgt nach dem **Request-Response Schema**. Der **HTTP Client** sendet seine Anfrage an den **HTTP Server**, der diese bearbeitet und eine Antwort zurücksendet. Im Gegensatz zum FTP-Protokoll sieht HTTP beim Verbindungsaufbau keine mehrstufige Handshake Phase vor. Weiterer Vorteil: HTTP ist grundsätzlich abwärtskompatibel. So können zum Beispiel Browser, die HTTP 1.1 beherrschen, auch mit HTTP-1.0 kompatiblen Webservern kommunizieren. Die jeweils höhere Versionsnummer muss sich bei diesem Vorgang automatisch an die ältere Version anpassen.

## 1. Geschichte

Die ursprüngliche Version HTTP 0.9 war als einfaches Transportprotokoll für Daten ausgelegt. Sie unterstützte lediglich die **GET-Methode**. MIME-Unterstützung für die Übertragung von Binärdaten oder ein Authentifizierungsmechanismus waren nicht vorgesehen.

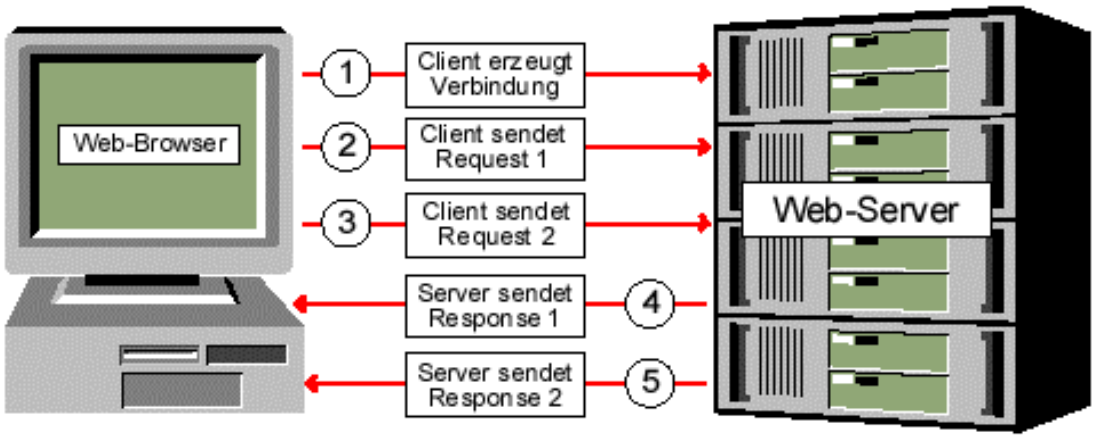
Mit HTTP 1.0 (RFC 1945) wurde ein erweitertes Request-Response-Format eingeführt, das die Übermittlung von mehr Informationen in beide Richtungen erlaubt. Der eigentlichen Request folgt ein Satz von Header-Feldern, die beispielsweise die Übermittlung des Browsertyps erlauben. Seit Version 1.0 kennt HTTP auch die POST-Methode, mit der zum Beispiel Formulareingaben vom Browser zum Webserver gelangen.

Weitreichendere Verbesserungen brachte die Einführung von HTTP 1.1. Eine der interessantesten Neuerungen: **persistente Verbindungen**. Während bei HTTP 1.0 jeder Request über eine neue Verbindung zum Server übertragen wird, können in der aktuellen Version Verbindungen aufrechterhalten werden, um mehrere Requests zu übermitteln. Über die neue Funktion Request Pipelining sendet ein Client nachfolgende Requests an den Server, bevor die Antwort des Servers auf die vorhergehende Anforderung auf der Clientseite eingetroffen ist. Im Zusammenspiel mit persistenten Verbindungen bringt das einen deutlichen Performancegewinn. Ebenfalls für mehr Performance sorgen die in HTTP 1.1 hinzugekommenen Cache-Kontrollfunktionen. HTTP 1.1 ist in RFC 2616 definiert

## 2. Ablauf einer HTTP Verbindung

Die Kommunikation zwischen Client und Webserver erfolgt durch den Austausch von HTTP Nachrichten. Diese Nachrichten übertragen die Anfragen und Antworten zwischen Client und Server. Client und Server bauen zum Austausch der Nachrichten im Standardfall eine TCP-Verbindung auf Port 80 auf. Die auch Request und Response genannten Nachrichten bestehen im Wesentlichen aus zwei Teilen: Header und Daten. Der Header enthält Steuerinformationen wie zum Beispiel die verwendete Methode und den gewünschten URL. Die HTTP-1.1-Spezifikation sieht insgesamt 46 zum großen Teil optionale Headereinträge vor. Diese sind in vier Kategorien unterteilt: Allgemeine Header-, Response-, Request- und Entity-Header-Einträge. Die allgemeinen Headerfelder sind sowohl in den Anfragen, als auch in den Antworten enthalten. Entity Header

beschreiben den Datenteil der Nachricht. Der Datenabschnitt der Nachricht selbst besteht meist aus einem HTML-Dokument oder Formulardaten, die der Client an den Server sendet.




Der Ablauf einer ***persistenten*** HTTP-Verbindung. Dabei wird eine Verbindung für mehrere Requests genutzt.

## 3. HTTP Requests

Ein Request ist durch die Angabe von Methode, URL und den Request-Header-Feldern bestimmt. Dabei kommen beispielsweise die Methoden GET, PUT oder DELETE zum Einsatz. Ein Server antwortet auf jeden Request mit Informationen, ob die gewählte Methode zulässig ist oder nicht.

Eine Methode für sich genommen ist ohne Angabe des Ziels wertlos. Daher gehört zu jeder Methode ein Ziel-URL. Der Client muss dabei einen absoluten URL angeben, damit der Request auch über einen Proxyserver laufen kann. Nach dem Zugriff auf eine Quelle reicht die Angabe von relativen URLs.

HTTP Request weist folgende Struktur auf:

```
METHOD URL HTTP/version
General Header
Request Headers
Entity Header (optional)
Leerzeile
Request Entity (falls vorhanden)
```

Ein Request, der eine HTML Seite anfordert, sieht beispielsweise so aus:

```
GET /verzeichnis1/seite2.html HTTP/1.1
Date Thursday, 14-Oct-99 17:55 GMT
User-agent: Mozilla/4.6
Accept: text/html, text/plain
```

Zuerst übermittelt der Client die Methode, durch Leerzeichen getrennt folgen der URL und die HTTP-Version. Die weiteren Felder übermitteln die Uhrzeit, die Browserversion und welche MIME-Typen der Client akzeptiert. Ein Datenbereich entfällt bei diesem Nachrichtentyp.

# 4. HTTP-Response

Der Aufbau eines HTTP Response ist ähnlich zum Request:

```
HTTP/version Status-Code Reason-Phrase
General Header
Response Header
Entity Header (optional)
Leerzeile
Resource Entity (falls vorhanden)
```

Ein kompletter Response, der eine HTML-Datei vom Server übermittelt, sieht z.B. so aus:

```
HTTP/1.1 200 OK
Via: HTTP/1.1 proxy_server_name
Server: Apache/1.3
Content-type: text/html
Content-length: 78

<html>
<head>
  <title>HTTP</title>
</head>
<body>
  <p>HTTP/1.1-Demo</p>
</body>
</html>
```

Der Server übermittelt zunächst die HTTP-Version der Nachricht. Der zweite Eintrag ist die Statusmeldung. "200 OK" bedeutet in diesem Fall, dass kein Fehler aufgetreten ist. Wichtig für die weitere Bearbeitung durch den Client sind die Einträge Content Type und Content Length. Content Type beschreibt den MIME-Typ der im Datenbereich übermittelten Datei. Im Header-Feld Content Length gibt der Server die Länge der Daten in Byte an. Der Einsatz des Feldes ist dabei nicht zwingend vorgeschrieben. Sollte das Feld fehlen, ist die Ermittlung der Datenlänge vom Typ der gesendeten Daten abhängig. Das RFC gibt daher die Empfehlung, das Feld immer zu senden.

## 4.1. Response-Codes

Die Antwort des HTTP-Servers beinhaltet die Statuszeile und Response-Header-Felder. Die Statuszeile wiederum führt die Protokollversion, den Status Code und Reason Phrase auf. Beim Status Code handelt es sich um einen dreistelligen Integer-Wert, der dem Client wichtige Informationen über Verfügbarkeit, erfolgreiche Bearbeitung oder aber auch Fehlermeldungen liefert. Die Reason Phrase enthält die Klartext-Bezeichnung der Meldung. Bekannte Fehlermeldungen sind beispielsweise 404 für "File not Found" (Datei nicht gefunden) oder 403 für "Forbidden" (Zugriff verweigert). Diese Meldungen sind in fünf Kategorien eingeteilt:

- **1xx**: Informelle Meldungen: Request erhalten, Bearbeitung wird durchgeführt.
- **2xx**: Erfolg: Request wurde erfolgreich erhalten, verstanden und angenommen.
- **3xx**: Weiterleiten: Weitere Aktionen müssen eingeleitet werden, damit eine Request vollständig bearbeitet werden kann.
- **4xx**: Clientfehler: Die Request enthält ungültigen Syntax oder kann nicht bearbeitet werden.
- **5xx**: Serverfehler: Der Server kann eine gültige Request nicht bearbeiten.

# 5. Detaillierter Ablauf einer Anfrage für ein HTML-File

Der HTTP-Standard 1.1 kann nicht ein HTML-File und alle darin vorhandenen Bilder gleichzeitig senden. Der Web-Browser geht also wie folgt vor:

1. Anfrage für ein HTML-File (z.B. `titel.html`)
2. Interpretierung des erhaltenen HTML-Files
3. Falls gewisse Tags weitere Files verlangen, wird eine neue Anfrage für die entsprechenden Files an den Webserver geschickt. (z.B. `<IMG SRC="logo.gif">`)

## 5.1. Die HTML-Testseite `hta_test.html`

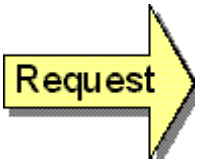
Die folgenden HTTP-Requests und HTTP-Response werden für die Seite `hta_test.html` hin- und hergeschickt. Die Seite `hta_test.html` beinhaltet ein Hintergrundbild und ein normales Bild, das im zweiten und dritten HTTP-Request angefordert wird. Die HTML-Seite `hta_test.html` sieht so aus:

```
<html>
<head><title>Anzeige von Bildern</title></head>
<body background="pictures/bg.gif">
<img src="pictures/logo.jpg">
</body>
</html>
```

## 5.2. HTTP-Kommunikation beim Laden von `hta_test.html`

Anforderung HTML Seite `demo/picture.html`

```
GET /demo/picture.html HTTP/1.1
Accept: */*
Accept-Language: de-ch
Accept-Encoding: deflate, gzip
User-Agent: Mozilla/4.0 (compatible; MSIE 5.01; Windows NT
5.0)
Host: localhost
Connection: Keep-Alive
```





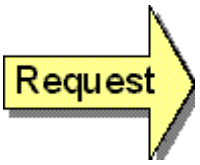


```
HTTP/1.1 200 OK
Server: ExperimentalWebServer 1.0
Content-type: text/html
Content-length: 143

<html>
<head><title>Anzeige von Bildern</title></head>
<body background="pictures/bg.gif">
<img src="pictures/logo.jpg">
</body>
</html>
```

Anforderung Hintergrundbild `bg.gif`

```
GET /demo/pictures/bg.gif HTTP/1.1
Accept: */*
Referer: http://localhost/demo/hta_test.html
Accept-Language: de-ch
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 5.01; Windows NT
5.0)
Host: localhost
Connection: Keep-Alive
```







```
HTTP/1.1 200 OK
Server: ExperimentalWebServer 1.0
Content-type: image/gif
Content-length: 153
```

*(binäre Daten...)*

Anforderung Bild `logo.jpg`

```
GET /demo/pictures/logo.jpg HTTP/1.1
Accept: */*
Referer: http://localhost/demo/hta_test.html
Accept-Language: de-ch
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 5.01; Windows NT
5.0)
Host: localhost
Connection: Keep-Alive
```







```
HTTP/1.1 200 OK
Server: ExperimentalWebServer 1.0
Content-type: image/jpeg
Content-length: 5140
```

*(binäre Daten...)*

# 6. HTTP Methoden

Jede Client Request wird durch die Angabe der Methode eingeleitet. Methoden bestimmen die Aktion der Anforderung. Die aktuelle HTTP-Spezifikation sieht acht Methoden vor: OPTIONS, GET, HEAD, POST, PUT, DELETE, TRACE und CONNECT.

### GET Methode

Die mit Abstand wichtigste Methode ist GET. Sie dient zur Anforderung eines Dokuments oder einer anderen Quelle. Eine Quelle wird dabei durch den Request-URL identifiziert. Man unterscheidet zwei Typen: conditional GET und partial GET. Beim Conditional GET Typ ist die Anforderung von Daten an Bedingungen geknüpft. Die genauen Bedingungen sind dabei im Header Feld "Conditional" hinterlegt. Oft gebrauchte Bedingungen sind zum Beispiel ***If-Modified-Since***, ***If-Unmodified-Since*** oder ***If-Match***. Mit Hilfe dieser Bedingung lässt sich die Netzbelastung deutlich verringern, da nur noch die wirklich benötigten Daten übertragen werden. In der Praxis nutzen zum Beispiel Proxyserver diese Funktion, um die mehrfache Übertragung von Daten, die sich bereits im Cache befinden, zu verhindern.

Das gleiche Ziel verfolgt die partielle GET Methode. Sie verwendet das Range-Header Feld, das nur Teile der Daten überträgt, die der Client jedoch noch verarbeiten kann. Diese Technik wird für die Wiederaufnahme eines unterbrochenen Datentransfers verwendet.

### POST Methode

Den umgekehrten Weg nimmt die POST Methode: Sie übermittelt in erster Linie Formulareingaben an einen Webserver. Aber auch die Kommentierung bestehender Quellen, Übermittlung von Nachrichten an Foren und Erweiterung von Online-Datenbanken sind mit POST möglich. Die an den Server übermittelten Daten sind in der Entity-Sektion enthalten. Auch die POST Methode übermittelt einen URL. In diesem Fall dient dieser lediglich als Referenz, welche Routine auf dem Server die Bearbeitung der Daten übernimmt.

### OPTIONS Methode

Über diese Methode kann der Client Informationen über verfügbare Kommunikationsoptionen abrufen. So lassen sich insbesondere Beschränkungen von Quellen auf einem HTTP Server oder auch einem Proxyserver ermitteln, ohne das eine bestimmte Aktion eingeleitet oder gar ein Datentransfer stattfindet.

### HEAD Methode

Diese Methode ist GET in seiner Funktionsweise sehr ähnlich. Einziger Unterschied: HEAD fordert lediglich den Header eines Dokuments oder Quelle an. Im Gegensatz zu GET übermittelt der Server aber nicht die eigentlichen Daten. HEAD eignet sich insbesondere dazu, die Größe von Quellen, Typ oder Objektattribute ausfindig zu machen. Der Server übermittelt auf eine HEAD Anfrage die Metainformationen, die identisch mit den Informationen der GET Request sind.

### PUT Methode

Dieser Typ erlaubt die Modifikation bestehender Quellen beziehungsweise Erzeugung neuer Daten auf dem Server. Im Unterschied zur POST Methode identifiziert der URL in der PUT Methode die mit der Anforderung gesendeten Daten selbst, und nicht die Quelle.

**DELETE Methode**

Mit Hilfe dieses Typs werden Daten auf dem HTTP Server gelöscht, die durch den URL identifiziert sind. Das Interessante dabei: Der Löschvorgang muss nicht unmittelbar nach dem Eingang der Anforderung, sondern kann auch zu einem späteren Zeitpunkt erfolgen. Der Server soll laut Spezifikation zumindest die Annahme des Requests bestätigen.

**TRACE Methode**

Über diese Methode kann der Client den Request verfolgen, die über mehrere Knotenpunkte laufen. Dies ist insbesondere bei der Übermittlung des Requests über einen oder mehrere Proxyserver interessant. Das letzte Glied der Kette generiert die Antwort. Die TRACE-Methode dient in erster Linie der Diagnose von Client-Server Verbindungen. Über das Max-Forwards-Header Feld bestimmt der Client die maximale Anzahl an Hops. Im Header Feld "Via" der Antwortnachricht sind alle durchlaufenen Server protokolliert.

**CONNECT Methode**

Die CONNECT Methode ist in der HTTP/1.1 Spezifikation für Verbindungen reserviert, bei denen Proxyserver dynamisch als Tunnel agieren. In der Praxis kann es sich beispielsweise um SSL Tunnel handeln. Der Tunnelmechanismus ist in der ersten Linie als Durchgang für SSL-gesicherte Verbindungen durch eine Firewall gedacht. In Proxyserver wie MS Proxy, Netscape Proxyserver oder auch WinGate ist diese Methode bereits implementiert. Der Client bestimmt durch die CONNECT-Methode samt Portangabe den Zielrechner. Der Proxyserver baut dann einen Tunnel zum angegebenen Rechner auf, und übermittelt Daten und Kommandos zwischen Client und Server. Details zu diesem Verfahren sind im Internet Draft " Tunneling TCP based protocols through Web proxy servers" festgelegt.

# 7. Authentifizierung über HTTP

Die HTTP Spezifikation sieht zwei parallele Authentifizierungsmechanismen vor: Der eine dient der Authentifizierung des Users beim HTTP-Server (Basic HTTP Authentication), der andere der Authentifizierung bei einem zwischengeschalteten Proxyserver (Proxy Server Authentication). Die Funktionsweise der beiden Mechanismen ist sehr ähnlich: Sie nutzen Request-, Response-Status-Code- und Response-Header. Unterschiede gibt es hingegen bei den verwendeten Status Codes und den Headernamen. Und: Die Authentifizierung eines Benutzers kann simultan bei zwischengeschalteten Proxyserver und dem HTTP Server erfolgen.

## 7.1. Basic HTTP Authentication

Das Basic-HTTP-Authentication-Schema verwendet Benutzerkennung und Passwort für die Authentifizierung. Der Client sendet mit seinem ersten Request eine Anfrage ohne Authentifizierungsinformationen. Die Antwort des Servers enthält das WWW-Authenticate-Header Feld, das den Client auffordert, die Zugangsdaten zu übermitteln. In diesem Headerfeld sind zudem Details definiert, wie die Authentifizierung erfolgen muss. Der Client wird zur Übermittlung der Authentifizierungsinformationen aufgefordert. Der Server überträgt in seiner Antwort den so genannten ***Realm***, eine Zeichenfolge, mit der er dem Client mitteilt, wer die Daten fordert. Der Client verwendet für die Kodierung von Kennung und Passwort das Base64 -Verfahren. Diese kodierte Zeichenfolge wird im Authorization-Header-Feld an den Server übermittelt. Ein Beispiel für eine solche Request:

```
GET /verzeichnis_x/seite_y.html HTTP/1.1
User-agent: Mozilla/4.6
Accept: text/html, image/gif, image/jpeg
Authorization: Basic KDWkfowsOm=
```

Der Authentifizierungsvorgang muss allerdings bei jedem Request erfolgen. Der Client legt daher die Daten im Cache ab und greift bei jedem nachfolgenden Request auf diese Informationen zurück.

## 7.2. Digest Access Authentication

Die Basic-HTTP-Authentication unterliegt einer massiven Einschränkung: ***Sie sieht keine Verschlüsselung der Benutzerdaten vor***. Um diese Schwäche zu beseitigen, wurde Digest Access Authentication (RFC 2617) eingeführt.

Digest Access Authentication benutzt mehrere Parameter für die Verschlüsselung. Die Verwendung von Zufallswerten ist dabei vorgeschrieben. Für die Verschlüsselung des Passwortes nimmt der Mechanismus neben dem Zufallswert den Benutzernamen, das Passwort, die HTTP Methode und die angeforderte URL. Die Verschlüsselungsmethode basiert in der Praxis meist auf einem MD5 -Algorithmus. Versucht ein Client auf eine gesicherte Site zuzugreifen, antwortet der Server zunächst mit einem Unauthorized Header:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Digest
realm="testrealm@host.com",
qop="auth,auth-int",
nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093",
opaque="5ccc069c403ebaf9f0171e9517f40e41"
```

Der Client übermittelt anschließend Benutzername und Passwort zum Server. Die Antwort des Clients verdeutlicht der nachstehende Authorization Header:

```
Authorization: Digest username="Benutzername",
realm="testrealm@host.com",
nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093",
uri="/dir/index.html",
qop=auth,
nc=00000001,
cnonce="0a4f113b",
response="6629fae49393a05397450978507c4ef1",
opaque="5ccc069c403ebaf9f0171e9517f40e41"
```

Nach der Übermittlung des verschlüsselten Passwortes gewährt oder aber verwehrt der Server den Zutritt. Die zwischen HTTP Server und Client übertragen Daten und Kommandos bleiben von diesem Mechanismus unberührt.